

SE LAB-4

NAME: PURVIK V

SRN:PES1UG24AM211

In this Pygame Defender game, fix the radar positioning bug.

The radar currently uses the player's screen-relative position for objects, which causes humanoids, landers, and the player to appear at incorrect positions on the radar when the camera scrolls.

Modify the existing code so that the radar represents the actual WORLD positions of the player, humanoids, and landers.

Requirements:

1. Do not remove or break any existing game functionality.
2. Keep the existing radar UI/design.
3. Use world coordinates when calculating radar positions.
4. Correctly convert world coordinates into radar coordinates.
5. Make sure the radar remains correct when the camera/player moves horizontally.
6. Keep the changes minimal and beginner-friendly.
7. Explain what you changed and why.
8. Give me the updated code for the affected function.

Absolutely. For Task 1, the minimal fix is to change the radar's X-coordinate calculation so it uses the world X coordinate directly, rather than `screen_x()`.

Change made

The problematic approach was:

```
</> Python
rx = self.screen_x(x) % VIEW_W
```

`screen_x()` converts world coordinates into camera-relative screen coordinates. That is appropriate for drawing objects in the game view, but not for a radar representing the whole world.

Use:

```
</> Python
rx = (x / WORLD_W) * VIEW_W
```

This maps:

```
</> Plain text
World X: 0 _____ WORLD_W
```

Now implement the second feature in the same Pygame Defender game.

Implement the `sky_color(wave)` function.

Requirements:

1. Return an RGB colour based on the current wave number.
2. The sky colour should change as the wave increases.
3. Keep the implementation simple and beginner-friendly.
4. Do not remove or break any existing functionality.
5. Use the existing game structure and make minimal changes.
6. Explain what you changed and why.
7. Give me the updated function code.

For Task 2, implement `sky_color(wave)` with a simple wave-based colour progression.

What changed

The function now returns a different RGB colour depending on the current wave. For waves beyond the defined colours, it keeps using the final colour.

Updated function

```
Python ↕ 📄 ▶ Run  
  
def sky_color(wave):  
    """Return an (r, g, b) sky colour for the current wave."""  
    colors = [  
        (5, 5, 20),      # Wave 1  
        (10, 15, 40),   # Wave 2  
        (15, 25, 60),   # Wave 3  
        (25, 15, 55),   # Wave 4  
        (40, 10, 45),   # Wave 5  
    ]  
  
    return colors[min(wave - 1, len(colors) - 1)]
```

This is a minimal change and works with the existing `draw()` logic, which already calls `sky_color(self.wave)`. No other game functionality needs to be changed.

Now implement the third feature in the same Pygame Defender game.

Implement the `on_humanoid_rescued(humanoid)` function.

Requirements:

1. The function should be called when a falling humanoid is successfully rescued.
2. Keep the existing +500 score behaviour.
3. Add simple feedback when a humanoid is rescued.
4. Do not break any existing game functionality.
5. Keep the implementation beginner-friendly and minimal.
6. Explain what you changed and why.
7. Give me the updated function code.

For Task 3, the existing game already handles the +500 score when a humanoid is rescued. The callback can therefore provide simple feedback without changing the scoring logic.

What changed

`on_humanoid_rescued(humanoid)` now prints a rescue message when the callback is triggered.

Updated function

 Python



 Run

```
def on_humanoid_rescued(humanoid):  
    """Called when the player catches a falling humanoid."""  
    print("Humanoid rescued! +500 points")
```

This keeps the implementation minimal and beginner-friendly, while preserving the existing rescue and scoring behaviour.

Now implement the final feature in the same Pygame Defender game.

Implement the `bonus_life_threshold()` function.

Requirements:

1. Return a score threshold at which the player earns an extra life.
2. Use a simple fixed score threshold.
3. The threshold should be 10000.
4. Make sure it works with the existing game logic.
5. Do not remove or break any existing functionality.
6. Keep the implementation beginner-friendly and minimal.
7. Explain what you changed and why.
8. Give me the updated function code.

For Task 4, the function can simply return the required fixed threshold of 10,000 points.

What changed

`bonus_life_threshold()` now returns `10000`. The existing game logic can use this value to determine when the player earns an additional life.

Updated function

 Python



 Run

```
def bonus_life_threshold():  
    """Return the score required to earn an extra life."""  
    return 10000
```